



ORCHESTRATING TRUST: A COMPOUND AGENTIC AI ARCHITECTURE FOR GROUNDED AND CONSTRAINT-AWARE TRAVEL PLANNING

By

Youssef Hesham

Yaseen El-Beltagy

Youssef Amr

Mahmoud Nazmy

A Thesis Submitted to the School of
Information Technology at New Giza University
in Partial Fulfillment of the
Requirements for the Degree of
BACHELOR OF SCIENCE
in
Information Technology
Under the Supervision of

Dr. Neamat El-Gayar

New Giza University

Department

Information Technology, New Giza University

Eng. Mohamed Abadallah

New Giza University

Department

Information Technology, New Giza University

INFORMATION TECHNOLOGY, NEW GIZA UNIVERSITY

GIZA, EGYPT
2026

Acknowledgements

Table of Contents

Acknowledgements	i
Table of Contents	ii
List of Tables	iv
List of Figures	v
Abstract	vi
1 INTRODUCTION	1
2 BACKGROUND	2
2.1 General Research Context	2
2.2 Literature Review	3
2.2.1 Compound AI Systems and System-Centric Design	3
2.2.2 Standardized Architectures for Agentic AI	3
2.2.3 Multi-Agent Collaboration via Conversation Programming	4
2.2.4 TravelPlanner: A Benchmark for Real-World Travel Planning with Language Agents	4
2.2.5 Reflexion: Language Agents with Verbal Reinforcement Learning	5
2.2.6 Gorilla: Large Language Model Connected with Massive APIs	5
2.2.7 Toolformer: Language Models Can Teach Themselves to Use Tools	6
2.2.8 The Role of Perceived Smart Tourism Technology Experience for Tourist Satisfaction, Happiness and Revisit Intention	6
2.2.9 Enhancing User Engagement through Adaptive UI/UX Design: A Study on Personalized Mobile App Interfaces	7
2.2.10 AI-Driven Customer Engagement in Tokopedia: Satisfaction Mediation on Chatbot and Recommendation System	7
2.2.11 How Perceived Motivations Influence User Stickiness and Sus- tainable Engagement with AI-Powered Chatbots	8
2.2.12 Intelligent Tourism Management System	8
2.2.13 LOCUS: A Mobile Tourism Application and Recommender System	9
2.2.14 The AI Tech Stack Model	10
2.2.15 Architecture of Intelligent Systems	10
2.2.16 Tabulated Comparison of Similarities and Differences Between Sources	11
3 METHODOLOGY	15
3.1 System Architecture Overview	15
3.2 The Presentation Layer (Mobile Client)	15
3.2.1 User Interface (UI) and Interaction Patterns	16
3.2.2 State Management and Optimistic UI	16

3.2.3	External Navigation Handoff	16
3.3	Ingestion & Routing Layer (The Gateway)	17
3.3.1	Security and Authentication	17
3.3.2	Efficiency via Semantic Caching	17
3.3.3	Intelligent Routing	17
3.4	The Agentic Orchestration Layer (The Brain)	17
3.4.1	Agent A: Cleo (The Persona Agent)	18
3.4.2	Agent B: The Planner (The Logic Agent)	18
3.4.3	Tool Execution Environment	18
3.4.4	The Reflexion Loop	18
3.5	The Ground Truth Layer (Data Persistence)	18
3.5.1	Relational and Geospatial Data	19
3.5.2	Vector Search and Hybrid Retrieval	19
3.6	Automated Data Pipeline	19
3.6.1	Ingestion, Enrichment, and Vetting	19
3.7	User Interface Design	19
3.7.1	Visual Discovery and Personalized Retrieval	20
3.7.2	Trust Calibration and The Ground Truth Interface	20
3.7.3	Dual-Agent Interaction Paradigms	20
3.7.3.1	The Conversational Interface (Cleo)	23
3.7.3.2	The Structured Planning Interface (The Planner)	23
4	RESULTS AND DISCUSSION	25
5	CONCLUSION	26
	References	27

List of Tables

2.1	Comparison of AI Architecture and Agentic Coding Research	11
2.2	Comparison of Smart Tourism Technology and User Engagement Research	12
2.3	Comparison of Intelligent Systems and Software Architecture Research	13

List of Figures

3.1	VOYO Compound Agentic AI System Architecture (C4 Level 3). The diagram illustrates the four primary layers: Presentation, Gateway, Agentic Orchestration, and Ground Truth Data, highlighting the flow from user intent to verified execution.	16
3.2	The Discovery Workflow. The Home Screen (Left) combines geospatial exploration with personalized vector recommendations. The Taxonomy (Center) and Catalogue (Right) views allow for structured, filter-based retrieval.	21
3.3	The Info Card (Ground Truth Interface). This view serves as a trusted data source, presenting verified hard constraints while providing direct context bridges to both the Planner Agent and the Cleo Chatbot. .	22
3.4	Agentic Interfaces. The unstructured chat interface (Left) handles intent and queries, while the structured timeline (Right) visualizes the logical output of the Planner Agent.	24

Abstract

Chapter 1

Introduction

Chapter 2

Background

2.1 General Research Context

The domain of automated travel planning and digital tourism assistance has evolved significantly alongside advancements in artificial intelligence and information retrieval. Historically, digital travel aids relied on static recommender systems and aggregator platforms that filtered databases based on explicit user keywords (e.g., location, price range). While effective for information retrieval, these systems lacked the semantic understanding required to construct coherent, personalized itineraries or handle complex, multi-constraint queries.

The advent of Large Language Models (LLMs) introduced a paradigm shift in how users interact with information systems. Models such as GPT-4 demonstrated remarkable proficiency in natural language understanding and generation, enabling conversational interfaces that could seemingly draft travel plans and answer cultural queries. However, the deployment of monolithic LLMs in safety-critical and fact-sensitive domains, such as travel, revealed inherent architectural limitations. Foremost among these is the phenomenon of “hallucination,” where models generate plausible but factually incorrect information—such as inventing non-existent transit routes or misstating the operating hours of historical sites—due to their probabilistic nature and lack of real-time grounding. Furthermore, standard LLMs struggle with multi-step reasoning and adherence to “hard constraints” (e.g., strictly optimizing a route for time while adhering to a fixed budget), often producing itineraries that are textually coherent but logistically infeasible.

To address these limitations, research has increasingly focused on Retrieval-Augmented Generation (RAG). RAG architectures decouple knowledge from the model’s weights, allowing the system to retrieve relevant documents from an external knowledge base to condition the generation process. This approach significantly improves factual accuracy by grounding model outputs in retrieved context. However, standard RAG is often insufficient for complex planning tasks that require more than just information retrieval; they require active reasoning, state management, and interaction with external environments.

This necessity has led to the emergence of Agentic AI and Compound AI Systems. Unlike passive chatbots, autonomous agents are characterized by their ability to perceive, reason, act, and reflect. Frameworks for agentic systems typically decompose the architecture into distinct modules: Profile (managing user identity and preferences), Memory (maintaining context over long horizons), Planning (breaking down complex goals into executable steps), and Action (executing tools such as API calls or database queries). Within this domain, techniques such as Chain-of-Thought (CoT) prompting enable models to decompose complex logical problems

into intermediate steps, while the ReAct (Reason + Act) paradigm allows agents to interleave reasoning traces with actions, creating a feedback loop where the model can observe the output of a tool (e.g., a weather API) before proceeding.

Recent literature suggests that state-of-the-art performance in complex domains is no longer achieved by scaling model parameters alone, but by engineering compound systems where multiple specialized agents and tools collaborate. These systems increasingly employ Reflexion mechanisms—self-correction loops where agents critique and iteratively refine their own outputs against defined constraints—to ensure reliability and safety in real-world applications.

2.2 Literature Review

2.2.1 Compound AI Systems and System-Centric Design

Zaharia et al. [1] address the diminishing returns of relying solely on scaling monolithic Large Language Models (LLMs) to achieve state-of-the-art results in complex tasks. The authors identify that while model scale improves general reasoning, it fails to solve system-level challenges such as hallucination, lack of up-to-date knowledge, and enforcement of hard constraints. To overcome this, they propose the paradigm of *Compound AI Systems*, which engineer solutions by orchestrating interacting components—including multiple agents, retrievers, and databases—rather than querying a single model.

Critically, the authors highlight that this paradigm allows for dynamic trade-offs between performance and cost, enabling systems to route simple queries to cheaper models (e.g., FrugalGPT) while reserving high-compute resources for complex reasoning. The work identifies the optimization of these systems as a distinct challenge, noting that unlike traditional neural networks, compound systems contain non-differentiable components (e.g., search engines) that require novel optimization frameworks such as DSPy. By analyzing industry breakthroughs including Alpha-Code 2 and Gemini, the study validates the architectural shift from *model-centric* to *system-centric* AI design as the dominant approach for maximizing future AI performance.

2.2.2 Standardized Architectures for Agentic AI

Wang et al. [2] provide a comprehensive survey aimed at standardizing the rapidly expanding field of agentic artificial intelligence. The authors identify a critical fragmentation in agent design, where similar capabilities are implemented under inconsistent terminologies and architectural assumptions. To address this, they propose a unified architectural blueprint comprising four core modules: Profile (defining agent identity and role), Memory (managing historical context and retrieval), Planning (goal decomposition and reasoning), and Action (tool and API execution).

The study further distinguishes between architectural structure and the mechanisms through which agent capabilities are acquired. Three primary capability acquisition strategies are identified: model fine-tuning, prompt engineering, and mechanism engineering (e.g., trial-and-error evolution). Through an analysis of applications across social and engineering domains, the survey demonstrates that

agents designed with this modular separation—when evaluated using both subjective and objective benchmarks—exhibit superior performance in simulating human-like decision-making compared to isolated LLM interactions.

2.2.3 Multi-Agent Collaboration via Conversation Programming

Wu et al. [3] investigate the limitations of single-agent systems in executing complex, multi-step workflows that require both reasoning and code execution. The authors observe that isolated LLMs often struggle with long-horizon context retention and self-correction. To address these limitations, they introduce *AutoGen*, a framework that enables structured multi-agent conversation as a mechanism for task execution.

Central to this framework is the paradigm of *Conversation Programming*, which combines natural language prompts with explicit programming logic to control inter-agent communication. The architecture separates execution and reasoning responsibilities by assigning code execution and user intent to a *User Proxy* agent, while an *Assistant* agent focuses on logical generation. Additionally, the framework supports dynamic group conversations managed by a supervisory agent that selects participants based on task context. Experimental results demonstrate that this collaborative architecture significantly reduces error rates in reasoning and coding tasks compared to zero-shot single-agent baselines, validating the effectiveness of role separation in improving system reliability.

2.2.4 TravelPlanner: A Benchmark for Real-World Travel Planning with Language Agents

Xie et al. [4] address the limitations of monolithic large language models in executing complex, real-world tasks that require reliability, factual grounding, and multi-step reasoning. The proposed paradigm decomposes intelligence into orchestrated components, combining a central LLM reasoning core with external tools, persistent memory, and structured data sources, coordinated through explicit control logic rather than prompt-only reasoning. The architecture emphasizes task-level planning, tool invocation, and verification loops, enabling systems to dynamically query databases, APIs, and symbolic modules during inference.

Empirical evaluation across heterogeneous benchmarks, including multi-hop question answering and planning-oriented tasks, demonstrated consistent performance gains over single-model baselines, with reported improvements in task success rate and factual accuracy, alongside a measurable reduction in hallucinated outputs. This framework directly underpins VOYO’s system design by justifying the agentic separation between Cleo (conversational reasoning) and the Planner Agent (logistics optimization), as well as the use of a grounded PostgreSQL/PostGIS backend as an authoritative data layer. By adopting a compound architecture, VOYO aligns with the study’s central claim that reliable AI applications emerge from controlled composition of models, tools, and verified data rather than reliance on standalone LLM inference.

2.2.5 Reflexion: Language Agents with Verbal Reinforcement Learning

Shinn et al. [5] introduced *Reflexion*, a verbal reinforcement learning framework that addresses the inefficiency of traditional reinforcement learning for large language model (LLM) agents operating in interactive environments. Instead of updating model parameters, the proposed approach enables agents to iteratively improve performance by generating and storing natural-language self-reflections derived from task-level feedback. The architecture decomposes the agent into three coordinated components: an *Actor* LLM for action generation, an *Evaluator* model that produces scalar or binary reward signals, and a *Self-Reflection* model that transforms these signals into actionable textual feedback maintained within an episodic memory buffer.

Empirical results demonstrate substantial performance gains across diverse domains, including a 22% absolute improvement in task completion on ALFWorld sequential decision-making tasks, a 20% accuracy increase on HotPotQA multi-hop reasoning, and a state-of-the-art 91% pass@1 accuracy on the HumanEval programming benchmark, surpassing GPT-4 baselines. This work provides direct academic justification for VOYO’s iterative self-correction loop and the inclusion of persistent agent memory within the Planner Agent, enabling learning from failed itineraries or suboptimal travel plans without retraining. Furthermore, Reflexion supports VOYO’s compound-agent design by demonstrating that reliable improvement in agent behavior emerges from structured feedback, reflection, and memory rather than monolithic prompt-based reasoning alone.

2.2.6 Gorilla: Large Language Model Connected with Massive APIs

Patil et al. [6] proposed *Gorilla*, a retrieval-aware, fine-tuned LLaMA-7B model designed to address the pervasive problem of hallucinated and incorrect API usage in large language models when interacting with external tools. The system combines instruction fine-tuning, self-instruct data generation, and an explicit document retrieval module to enable accurate selection and invocation of APIs from a large, evolving toolspace. Gorilla is trained on *APIBench*, a curated benchmark containing 1,645 real-world APIs and 16,450 instruction-API pairs sourced from TorchHub, TensorFlow Hub, and HuggingFace, with correctness evaluated via AST sub-tree matching to distinguish functional errors from true hallucinations.

Empirically, Gorilla achieves 59.13% zero-shot accuracy on TorchHub APIs compared to 38.70% for GPT-4, while reducing hallucination rates to 6.98% versus GPT-4’s 36.55%, and reaches over 91% accuracy with oracle retrieval on HuggingFace tasks. These results provide direct justification for VOYO’s tool-centric agent architecture, where LLM reasoning is tightly coupled with verified external functions (e.g., routing, weather, and POI queries) rather than free-form text generation. In particular, Gorilla’s retriever-aware training motivates VOYO’s use of a grounded PostgreSQL/PostGIS knowledge layer and explicit tool interfaces for itinerary planning, ensuring that Cleo and the Planner Agent invoke only validated operations, thereby minimizing hallucinations and maintaining robustness as travel data and APIs evolve.

2.2.7 Toolformer: Language Models Can Teach Themselves to Use Tools

Schick et al. [7] introduced *Toolformer*, a self-supervised framework that enables large language models to autonomously learn when and how to invoke external tools via natural-language API calls, directly addressing hallucination, numerical reasoning errors, and lack of temporal awareness in standalone LLMs. The method augments a pretrained GPT-J (6.7B) model by sampling candidate API calls (e.g., search, calculator, calendar, translation), executing them, and retaining only those calls that reduce the future-token cross-entropy loss, thereby ensuring that tool usage is utility-driven rather than prompt-engineered. The resulting model is fine-tuned on an augmented corpus where tool calls are interleaved with text, allowing inference-time interruption, tool execution, and response injection.

Empirically, Toolformer achieves substantial zero-shot gains across multiple benchmarks, including an improvement from 17.8% to 33.8% on SQuAD-style factual completion, from 7.5% to 40.4% on ASDiv mathematical reasoning, and from 3.9% to 27.3% on DATESET temporal reasoning, frequently outperforming much larger models such as GPT-3 (175B). This work provides a direct methodological foundation for VOYO’s tool-centric agent design, where Cleo and the Planner Agent explicitly invoke verified functions (e.g., routing, hours lookup, weather, POI queries) instead of relying on implicit reasoning. Moreover, Toolformer’s loss-based filtering principle strongly motivates VOYO’s use of a grounded PostgreSQL/PostGIS backend as a deterministic tool layer, ensuring that itinerary generation and travel guidance are derived from validated data sources rather than free-form generation, thereby minimizing hallucinations in safety-critical tourism contexts.

2.2.8 The Role of Perceived Smart Tourism Technology Experience for Tourist Satisfaction, Happiness and Revisit Intention

Pai et al. [8] developed a structural equation modeling (SEM) framework to quantify how perceived Smart Tourism Technology (STT) attributes influence travel satisfaction, tourist happiness, and revisit intention. The study operationalized STT experience as a second-order construct composed of five first-order dimensions: informativeness, accessibility, interactivity, personalization, and security, validated using survey data from $N = 527$ tourists in Macau. Using PLS-SEM with bootstrapping (5,000 samples), all five attributes were found to significantly contribute to perceived STT experience, with accessibility exhibiting the strongest effect ($\beta = 0.285$, $T = 35.093$), followed by informativeness ($\beta = 0.254$) and security ($\beta = 0.239$).

Perceived STT experience demonstrated a strong positive impact on travel satisfaction ($\beta = 0.69$), which in turn significantly influenced tourist happiness ($\beta = 0.804$) and revisit intention ($\beta = 0.737$), while happiness independently affected revisit intention ($\beta = 0.475$), with all paths significant at $p < 0.001$. These findings provide empirical justification for VOYO’s design priorities, particularly the emphasis on high accessibility (mobile-first interaction, low-friction querying), information reliability via a grounded PostgreSQL backend, and security-aware data handling. Moreover, the validated mediation effect between satisfaction and revisit intention supports VOYO’s focus on an integrated, end-to-end agentic experience—where

Cleo’s conversational guidance and the Planner Agent’s itinerary optimization jointly enhance user satisfaction, ultimately reinforcing long-term engagement and reuse of the platform.

2.2.9 Enhancing User Engagement through Adaptive UI/UX Design: A Study on Personalized Mobile App Interfaces

Liu et al. [9] proposed a comprehensive adaptive UI/UX framework aimed at enhancing user engagement in mobile applications through real-time personalization driven by user profiles and contextual awareness. The system is architected as a modular pipeline consisting of a *User Profile Module*, a *Context-Aware Adaptation Engine*, a *Personalized Interface Generator*, a *User Engagement Metrics Tracker*, coordinated via a centralized data management layer. Methodologically, the framework integrates unsupervised learning (Self-Organizing Maps for user clustering), a multi-layer perceptron for contextual state inference, and reinforcement learning to iteratively optimize interface configurations based on observed engagement feedback.

Empirical evaluation was conducted through a large-scale deployment in a mobile learning application, involving over 10,000 active users and controlled usability testing with $N = 32$ participants. Results indicate a 22% increase in task completion rates, a 27% increase in session duration, a 35% improvement in feature discovery, and an average 14.8-point increase in System Usability Scale (SUS) scores compared to non-adaptive interfaces, with only marginal performance overhead ($< 11\%$ increase in battery consumption). This work provides a direct empirical foundation for VOYO’s adaptive front-end design, justifying the use of context-aware personalization in the user journey layer. In particular, the separation between Cleo’s conversational intelligence and VOYO’s dynamic UI rendering aligns with the paper’s finding that engagement gains are maximized when reasoning, personalization logic, and presentation layers are decoupled, reinforcing VOYO’s agentic architecture and data-driven UX optimization strategy.

2.2.10 AI-Driven Customer Engagement in Tokopedia: Satisfaction Mediation on Chatbot and Recommendation System

Christina et al. [10] empirically examined the role of AI-driven chatbots and recommendation systems in shaping customer engagement, explicitly modeling customer satisfaction as a mediating construct within an extended UTAUT framework. Using Partial Least Squares Structural Equation Modeling (PLS-SEM) on survey data from $N = 204$ active Tokopedia users, the study operationalized AI interaction through a conversational chatbot (“TANYA”) and an algorithmic recommendation engine, mapping them to performance expectancy, effort expectancy, and facilitating conditions.

Empirical results indicate that neither chatbot interaction ($\beta = 0.078$, $p = 0.402$) nor recommendation systems ($\beta = 0.136$, $p = 0.158$) directly increased engagement; however, both significantly improved satisfaction ($\beta = 0.445$ and $\beta = 0.450$, respectively, $p < 0.001$), which in turn exerted a strong positive effect on engagement

($\beta = 0.525$, $p < 0.001$). Mediation analysis confirmed that satisfaction fully mediated the effects of both AI components on engagement ($\beta = 0.234$ and $\beta = 0.237$, $p < 0.001$). These findings provide direct theoretical grounding for VOYO’s experience-first agentic design, demonstrating that AI systems influence sustained user engagement indirectly through perceived satisfaction rather than raw functional capability. Consequently, this study supports VOYO’s separation of Cleo as a conversational trust-building agent and the Planner Agent as a utility-focused decision engine, emphasizing that reliable itineraries and recommendations must be embedded within a satisfying, low-friction interaction layer to achieve long-term engagement in AI-powered tourism platforms.

2.2.11 How Perceived Motivations Influence User Stickiness and Sustainable Engagement with AI-Powered Chatbots

Pang et al. [11] developed a multi-theoretical explanatory model to address the problem of low long-term engagement and weak user stickiness in AI-powered chatbots, shifting the focus from initial adoption to sustainable usage behavior. The proposed framework integrates the Uses and Gratifications (U&G) Theory, Technology Acceptance Model (TAM), Expectation–Confirmation Model (ECM), and the Information Systems Success Model (ISSM), positioning user attitude as the primary mediating construct linking user motivations to behavioral outcomes. Using Structural Equation Modeling (SEM) on survey data from $N = 735$ active chatbot users, the study operationalizes four motivational dimensions—utilitarian, hedonic, technological, and social—alongside perceived privacy invasion.

Empirical results demonstrate that all four motivational factors positively influence user attitude, with technology motivation exhibiting the strongest effect ($\beta = 0.326$, $p < 0.001$), while privacy invasion negatively impacts attitude ($\beta = -0.094$, $p < 0.05$). User attitude was found to strongly predict both user stickiness ($\beta = 0.768$, $p < 0.001$) and sustainable usage ($\beta = 0.676$, $p < 0.001$), with stickiness further reinforcing sustainable usage ($\beta = 0.344$, $p < 0.01$). These findings provide a rigorous behavioral justification for VOYO’s experience-centric, compound AI architecture, supporting the separation of Cleo as a socially engaging conversational agent from the Planner Agent as a goal-oriented decision module. Moreover, the demonstrated negative impact of privacy invasion directly motivates VOYO’s use of a grounded PostgreSQL-backed knowledge layer and controlled data flows to preserve user trust, thereby aligning system architecture with empirically validated determinants of long-term engagement.

2.2.12 Intelligent Tourism Management System

Onuiri et al. [12] presented an Intelligent Tourism Management System (ITMS) to address the problem of information overload and fragmented access to tourism data in developing-country contexts, with a specific focus on Nigerian tourism. The proposed system adopts a web-based, centralized architecture designed using the Rational Unified Process (RUP), integrating a relational MySQL database with PHP and HTML front-end components to manage tourist locations, user profiles, bookings, and feedback. A key technical contribution is the incorporation of a hybrid

recommendation technique that filters and recommends tourist destinations based on user preferences, enabling basic personalization to support decision-making.

The system maintains a curated knowledge base of 50 tourist locations across Nigeria and supports core functionalities such as keyword-based search, location filtering by state, user ratings, and online reservation requests. Functional validation demonstrated that the platform successfully reduced search effort, improved information accessibility, and provided context-relevant recommendations, while remaining compatible across multiple browsers and operating systems. Although limited in scale and lacking advanced learning-based intelligence, this work provides an early architectural foundation for VOYO by highlighting the necessity of a centralized, structured tourism database coupled with recommendation logic. VOYO extends this concept by replacing static content retrieval with a grounded PostgreSQL/PostGIS backend and augmenting rule-based recommendation with agentic LLM reasoning, enabling dynamic itinerary generation, richer personalization, and conversational access to tourism data at a national scale.

2.2.13 LOCUS: A Mobile Tourism Application and Recommender System

AlSaeed [13] proposed *LOCUS*, a mobile tourism application with an integrated personalized recommender system, addressing the problem of information overload and lack of personalization in tourism-related mobile platforms. The system adopts a client-server architecture and implements a hybrid collaborative filtering approach, combining item-item and user-user collaborative filtering to recommend places and activities based on user characteristics, behavior history, and contextual attributes such as age, gender, and location. Recommendation similarity is computed using cosine similarity between feature vectors representing user preferences and activity attributes, with a predefined similarity threshold to trigger recommendations.

The application integrates external data sources via the Google Places API, while user data and activity metadata are stored in a centralized database, enabling continuous preference learning over time. Empirical evaluation was conducted through User Acceptance Testing (UAT) with $N = 10$ participants performing 17 representative tasks. Results indicate that approximately 70% of tasks were completed without errors, with an average error rate below 0.4 per task, significantly outperforming the accepted benchmark of 0.7. Efficiency analysis showed an average task completion time of 5.4s for most tasks, while usability assessment using the System Usability Scale (SUS) yielded a high score of 87.75, well above the standard acceptance threshold of 68. This work provides a concrete baseline for VOYO by demonstrating the effectiveness of structured tourism databases combined with recommendation logic in improving usability and satisfaction. VOYO extends this approach by replacing static collaborative filtering with agentic LLM-driven reasoning, integrating a grounded PostgreSQL/PostGIS backend, and enabling conversational access through Cleo, thereby supporting dynamic itinerary generation, richer personalization, and scalable deployment in complex tourism environments such as Egypt.

2.2.14 The AI Tech Stack Model

Tsaih et al. [14] proposed a seven-layer AI tech-stack model to address the management and technological complexity of developing AI-enabled applications, particularly the issues of vendor lock-in, poor interoperability, and tight coupling in existing AI-as-a-Service (AIaaS) offerings. The model decomposes AI systems into loosely coupled layers—AI Infrastructure, AI Platform, AI Framework, AI Algorithm, AI Data Pipeline, AI Services, and AI Solution—each with clearly defined responsibilities, enabling independent evolution and replacement of components.

Methodologically, the study adopts a mixed integration approach and complements the tech-stack with a synthesized IT/IS framework that evaluates organizational connection capabilities, incumbent IT/IS capabilities, and AI capabilities to guide in-house versus outsourcing decisions. The framework is empirically illustrated through four smart tourism case studies conducted with major Taiwanese travel companies, demonstrating how the layered architecture facilitates modular adoption of AIaaS components (e.g., MLOps platforms, recommendation APIs, data pipelines) while reducing migration cost and system fragility. Results show that organizations applying the model were able to clearly localize technical debt, define layer-specific SLAs, and flexibly integrate external APIs and data sources without disrupting higher-level application logic. This work provides direct architectural justification for VOYO’s compound, layered system design, particularly the separation between LLM-based reasoning agents (Cleo and the Planner), the grounded PostgreSQL/PostGIS data layer, and external AI services and APIs. By aligning VOYO with a loosely coupled AI stack, the system inherits the model’s proven benefits in scalability, interoperability, and long-term maintainability, which are critical for a national-scale, AI-powered tourism platform.

2.2.15 Architecture of Intelligent Systems

Swanepoel et al. [15] proposed a reference architecture for intelligent tourism systems to address the challenges of scalability, interoperability, and data heterogeneity in smart tourism platforms. The architecture is grounded in a layered, service-oriented design, separating concerns across data acquisition, data management, intelligence and analytics, and application services, with explicit support for integrating heterogeneous data sources such as IoT sensors, open tourism datasets, third-party APIs, and user-generated content.

Methodologically, the framework emphasizes semantic data modeling, context-awareness, and modular AI components, enabling intelligent services such as personalized recommendations, real-time decision support, and adaptive user interaction. The proposed architecture is validated through comparative analysis with existing smart tourism and smart city systems, demonstrating improved extensibility, clearer separation of responsibilities, and reduced coupling between data, intelligence, and presentation layers. This work provides architectural grounding for VOYO’s system-level design by reinforcing the importance of a decoupled, layered architecture in intelligent tourism applications. In particular, it supports VOYO’s separation between grounded data management (PostgreSQL/PostGIS), agentic intelligence layers (Cleo and the Planner Agent), and application-facing services, ensuring that reasoning, data access, and user interaction evolve independently while maintaining

system robustness and scalability at a national tourism scale.

2.2.16 Tabulated Comparison of Similarities and Differences Between Sources

The following tables categorize the primary research sources based on their technical contributions, methodologies, and specific roles within the VOYO project.

Table 2.1: Comparison of AI Architecture and Agentic Coding Research

Research Source	Primary Focus	Grounding & Tool-Use	Technical Goal	Role in VOYO
1. Compound AI Systems [1]	Orchestrating interacting components (agents/DBs) rather than scaling models.	Uses retrievers and databases to solve model hallucinations.	Shift from model-centric to system-centric AI design.	Defends against “wrapper” criticism; justifies a heavy backend.
2. Autonomous Agent Survey [2]	Standardizing agent modules: Profile, Memory, Planning, and Action.	Provides a framework linking the “brain” to the tool suite.	Unifying inconsistent terminologies into a modular blueprint.	Maps VOYO components: Cleo (Profile) and Vector Store (Memory).
3. Auto-Gen [3]	Enabling task solving via structured multi-agent conversations.	Separates reasoning (Assistant) from execution (User Proxy) to reduce errors.	Proving that separation of concerns improves reliability in complex workflows.	Justifies separating Cleo (conversation) from the Planner (logistics).
4. TravelPlanner [4]	Solving multi-constraint logistical travel challenges (time, cost, pace).	Uses “ground truth” content for accurate, safe, and efficient travel recommendations.	Ensuring itinerary schedules are logical and feasible.	Validates the use of specialized agents to verify logistics via tools.

Continued on next page

Table 2.1: Comparison of AI Architecture and Agentic Coding Research (Continued)

Research Source	Primary Focus	Grounding & Tool-Use	Technical Goal	Role in VOYO
5. Gorilla [6]	Optimizing external API calls and documentation accuracy.	Essential for implementing APIs like Mapbox and OpenWeatherMap.	Ensuring API requests do not fail due to formatting or hallucinated parameters.	Provides technical reliability to outperform “thin wrapper” competitors.
6. Tool-former [7]	Enabling LLMs to self-decide when and how to use external tools.	Deciding if a user query requires a tool call (e.g., database) or just a response.	Exploring self-supervised methods for tool integration.	Provides the intelligence to discover “hidden gems” through database cross-referencing.
7. Reflexion / Agentic Survey [5]	Structural roadmap for autonomous agents focusing on planning and memory.	Systematic framework for linking LLM reasoning to curated knowledge graphs.	Defining architectural principles for trusted AI companions.	Academic backbone for agents with specific system prompts and goals.

Table 2.2: Comparison of Smart Tourism Technology and User Engagement Research

Research Source	Primary Focus & Theory	Methodology & Sample	Key Findings & Results	Role in VOYO
8. Smart Tourism Tech [8]	Quantifying how STT attributes drive satisfaction and happiness.	PLS-SEM with 5,000 bootstrapping samples ($N = 527$ tourists).	Accessibility is the strongest driver ($\beta = 0.285$); satisfaction predicts revisit intention ($\beta = 0.737$).	Justifies high accessibility goals and grounded data for long-term retention.

Continued on next page

Table 2.2: Comparison of Smart Tourism Technology and User Engagement Research (Continued)

Research Source	Primary Focus & Theory	Methodology & Sample	Key Findings & Results	Role in VOYO
9. Adaptive UI/UX Design [9]	Enhancing engagement through real-time interface personalization.	Modular pipeline using RL; tested on 10,000+ active users.	22% increase in task completion and 35% improvement in feature discovery.	Empowers VOYO's adaptive front-end and decoupled presentation layers.
10. AI-Driven Engagement [10]	Role of chat-bots/recommenders in engagement via UTAUT framework.	PLS-SEM on survey data from active users ($N = 204$).	Satisfaction fully mediates the effect of AI on engagement; functionality alone is insufficient.	Supports the separation of "Cleo" (trust) and "Planner" (utility) agents.
11. User Stickiness & Motivations [11]	Linking utilitarian, hedonic, and tech motivations to sustainable usage.	SEM integrating U&G and TAM on chatbot users ($N = 735$).	Technology motivation is the strongest driver ($\beta = 0.326$); privacy concerns hinder engagement.	Validates VOYO's data protection strategies to maintain user trust.

Table 2.3: Comparison of Intelligent Systems and Software Architecture Research

Research Source	Primary Focus	Methodology & Architecture	Key Contributions & Results	Role in VOYO
12. Intelligent Tourism Management System [12]	Addressing information overload and fragmented tourism data access.	Web-based centralized architecture using the Rational Unified Process (RUP) and MySQL.	Implemented hybrid recommendation techniques; successfully reduced search effort and improved accessibility.	Highlights the necessity of a centralized, structured tourism database for VOYO's grounded backend.

Continued on next page

Table 2.3: Comparison of Intelligent Systems and Software Architecture Research
(Continued)

Research Source	Primary Focus	Methodology & Architecture	Key Contributions & Results	Role in VOYO
13. LOCUS: Mobile Tourism & Recommender [13]	Personalized recommendations to combat information overload on mobile.	Client-server architecture using hybrid collaborative filtering and cosine similarity.	Achieved a high SUS score of 87.75 and an average task completion time of 5.4s.	Demonstrates the effectiveness of linking mobile clients to structured server-side logic.
14. The AI Tech Stack Model [14]	Managing technological complexity and preventing vendor lock-in.	Proposed a seven-layer AI tech-stack (Infrastructure to Solution) using loosely coupled layers.	Organizations using the model localized technical debt and reduced system fragility.	Justifies VOYO's compound, layered design and the separation of LLM agents from data layers.
15. Architecture of Intelligent Systems [15]	Solving challenges of scalability and data heterogeneity in smart tourism.	Service-oriented design separating data management, intelligence, and application services.	Validated through comparative analysis, showing improved extensibility and reduced coupling.	Reinforces the importance of a decoupled architecture for national-scale tourism applications.

Chapter 3

Methodology

The VOYO system methodology represents a multi-layered technical strategy designed to bridge the “critical information gap” that currently leaves foreign travelers in Egypt vulnerable to tourist traps, scams, and fragmented logistics. While Generative AI offers a promising interface for natural language interaction, monolithic Large Language Models (LLMs) suffer from inherent stochasticity, often leading to hallucinations and a failure to adhere to hard constraints (e.g., specific opening hours or budget limits).

To mitigate these limitations, this project implements a **Compound Agentic AI Architecture** [1]. This approach shifts the focus from model scaling to system engineering, orchestrating multiple specialized components—including vector memory, relational databases, and distinct agentic roles—to serve as a reliable, 24/7 intelligent local guide. By decoupling the reasoning engine from the knowledge base, VOYO achieves a deterministic grounding in real-world data while retaining the conversational flexibility of modern LLMs.

The following sections detail the end-to-end methodology of the system, traversing from the latency-masking strategies in the presentation layer to the hybrid search algorithms in the persistence layer.

3.1 System Architecture Overview

The VOYO architecture is designed as a modular, backend-centric system. It adopts a micro-services pattern where the mobile client acts as a thin presentation layer, while the complex computational tasks of multi-agent orchestration, state management, and data verification are handled asynchronously by the server.

3.2 The Presentation Layer (Mobile Client)

The front-end of VOYO is developed using **Flutter**, a cross-platform framework selected for its high-performance Skia rendering engine. Unlike web-based wrappers, Flutter compiles to native ARM code, ensuring consistent 60fps performance on both iOS and Android devices. This native performance is critical for rendering complex map vector tiles without the jitter often associated with hybrid frameworks [9].

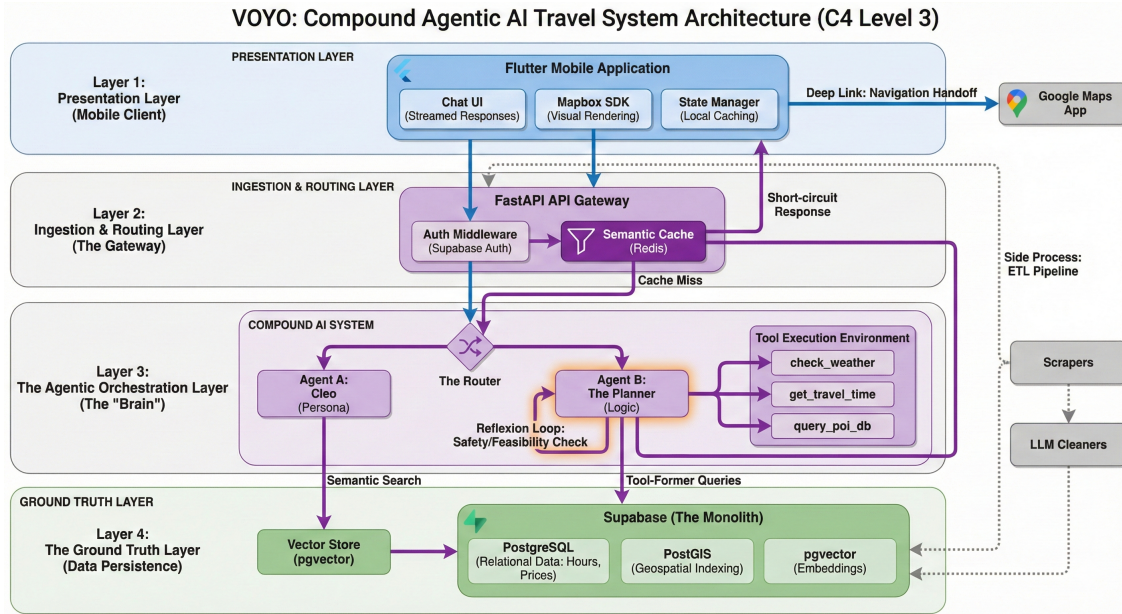


Figure 3.1: Voyo Compound Agentic AI System Architecture (C4 Level 3). The diagram illustrates the four primary layers: Presentation, Gateway, Agentic Orchestration, and Ground Truth Data, highlighting the flow from user intent to verified execution.

3.2.1 User Interface (UI) and Interaction Patterns

The application interface is bifurcated into two primary interaction modes. First, the **Chat UI** provides a natural language interface capable of handling Server-Sent Events (SSE). This allows the application to stream tokens from the LLM in real-time, reducing the "Time to First Byte" (TTFB) perception and maintaining user engagement during lengthy inference tasks. Second, the **Interactive Map Explorer**, powered by the Mapbox SDK, serves as the visual grounding for the AI's recommendations. By tightly coupling the chat stream with map markers, the system provides immediate visual context to textual suggestions.

3.2.2 State Management and Optimistic UI

A significant challenge in Agentic AI is latency; multi-step reasoning chains can take several seconds to execute. To mitigate the friction this causes, the client implements an **Optimistic UI State** pattern. When a user performs an action—such as adding a recommended restaurant to their itinerary—the State Manager immediately updates the local UI to reflect success, masking the network round-trip time. This state is managed via a reactive provider pattern, ensuring that if the backend validation fails (e.g., the restaurant is permanently closed), the UI can seamlessly roll back to the previous valid state and notify the user.

3.2.3 External Navigation Handoff

While Voyo handles the high-level logic of itinerary planning (the "what" and "when"), it delegates the "how" of physical transit. The application utilizes a **Deep Link Handoff** strategy, constructing precise URI schemes (e.g.,

`google.navigation:q=lat,lng`) to pass the final route coordinates to native navigation applications. This ensures users benefit from real-time traffic telemetry and turn-by-turn navigation without bloating the VOYO application size.

3.3 Ingestion & Routing Layer (The Gateway)

All client communication is managed by a high-performance **FastAPI** Gateway. This Python-based runtime was selected for its native support for asynchronous concurrency (`asyncio`). Given that agentic workflows utilize blocking I/O operations (such as waiting for external API responses), FastAPI’s event loop allows the server to handle thousands of concurrent connections without thread starvation, a critical requirement for scaling agentic systems.

3.3.1 Security and Authentication

Security is enforced via an Authentication Middleware integrated with **Supabase Auth**. This creates a stateless security layer where every request header is validated against a JSON Web Token (JWT). This ensures that all interactions are cryptographically signed and linked to a specific user profile (UUID), preventing unauthorized access to personalized itinerary data or interaction history.

3.3.2 Efficiency via Semantic Caching

To mitigate the high latency and inference costs associated with LLM calls, the gateway implements a **Semantic Cache** using Redis. Unlike traditional exact-match caching, semantic caching employs vector embeddings to identify requests that are conceptually similar. For example, queries like “Weather in Cairo” and “What is the temperature in Cairo?” map to proximal points in the vector space. By calculating the cosine similarity between the incoming query embedding and cached keys, the system can serve a pre-computed “short-circuit response” for high-frequency queries, bypassing the expensive AI layer entirely.

3.3.3 Intelligent Routing

In the event of a cache miss, the request is passed to **The Router**. This component acts as a zero-shot intent classifier. It analyzes the semantic content of the user’s query to determine the necessary computational path. Simple navigational queries may be routed to a keyword search, while complex planning requests containing temporal or budgetary constraints are directed to the appropriate specialized agent in the orchestration layer [3].

3.4 The Agentic Orchestration Layer (The Brain)

This layer represents the core innovation of the VOYO Engine. Rather than relying on a single general-purpose model, the architecture implements a multi-agent system [2] where distinct open-source LLMs (specifically optimized variants of Llama 3) act as reasoning engines equipped with specialized tools.

3.4.1 Agent A: Cleo (The Persona Agent)

Cleo acts as the front-line conversational interface. Its system prompt is engineered to enforce a specific persona: an empathetic, knowledgeable local guide. Cleo utilizes **Retrieval-Augmented Generation (RAG)** to ground its responses. When a user asks a cultural or historical question, Cleo performs a semantic search against the vector store to retrieve expert-vetted context chunks. These chunks are injected into the context window, allowing Cleo to answer with high factual accuracy while maintaining a consistent personality [10].

3.4.2 Agent B: The Planner (The Logic Agent)

Addressing the specific challenges of travel planning identified by Xie et al. [4], the Planner Agent is a logic-focused module stripped of conversational fluff. It is responsible for handling hard constraints such as budgets, operating windows, and geographical distances. The Planner operates using **Chain-of-Thought (CoT)** reasoning, explicitly decomposing complex requests (e.g., "Plan a 3-day trip to Luxor") into discrete logical steps: identifying candidate POIs, clustering them by location, and scheduling them within operating hours.

3.4.3 Tool Execution Environment

To prevent hallucination, the Planner functions within a **Tool Execution Environment** inspired by the Toolformer paradigm [7]. The agent is not permitted to rely on its internal parametric knowledge for volatile data. Instead, it is programmatically connected to a suite of deterministic tools—such as `check_weather`, `get_travel_time`, and `query_poi_db`. The Large Language Model functions as the controller, emitting API calls which are executed by the Python runtime [6]. The results are then fed back into the model’s context, ensuring that all assertions are verified against real-world data.

3.4.4 The Reflexion Loop

A critical safety mechanism within the architecture is the **Reflexion Loop** [5]. Before presenting any generated itinerary to the user, the system performs a self-correction pass. A specialized validator prompt critiques the proposed schedule against strict feasibility rules (e.g., "Is the museum actually open at the scheduled time?", "Is the travel time between stops realistic?"). If an error is detected, the agent internally iterates and refines the plan, transforming the output from a probabilistic guess into a verified schedule.

3.5 The Ground Truth Layer (Data Persistence)

The reliability of the entire system rests on the “Ground Truth” layer, implemented using **Supabase** (The Monolith). This unifies three distinct data paradigms into a single backend service to support the diverse needs of the agents.

3.5.1 Relational and Geospatial Data

Standard structured data, such as site names, opening hours, and pricing, is stored in normalized **PostgreSQL** tables. To support the map explorer and proximity-based queries, the system utilizes **PostGIS**, a spatial database extender. This allows the Planner Agent to execute efficient K-Nearest Neighbor (KNN) queries (e.g., “Find all safe restaurants within a 500-meter radius of the user’s current coordinates”), ensuring spatial coherence in the generated itineraries.

3.5.2 Vector Search and Hybrid Retrieval

To enable semantic understanding, high-dimensional embeddings of site descriptions and user reviews are stored using **pgvector**. This allows the AI agents to perform similarity searches based on abstract concepts (e.g., “quiet romantic dinner”) rather than just keywords. VOYO employs a **Hybrid Search** strategy, combining the precision of keyword matching (BM25) with the semantic breadth of vector search to maximize retrieval recall and relevance.

3.6 Automated Data Pipeline

To ensure the system remains a “single source of truth,” a continuous background pipeline maintains database integrity.

3.6.1 Ingestion, Enrichment, and Vetting

An automated **ETL Pipeline** utilizes headless browser scrapers to harvest raw data from external sources such as OpenTripMap and Google Places. This raw data is passed through an LLM-based **Enrichment Agent**, which cleans the text, summarizes descriptions, and applies the VOYO taxonomy. Crucially, every entry is assigned a **scam_risk** and **authenticity_score**. These synthetic metrics allow the Planner Agent to programmatically prioritize “hidden gems” and filter out high-risk tourist traps, directly addressing the trust deficit in the current tourism market.

3.7 User Interface Design

The User Interface (UI) of VOYO is not merely a presentation layer, but the functional bridge between the user and the Compound AI architecture. The design philosophy prioritizes **functional minimalism** and **trust calibration**. Recognizing that travel planning involves managing high-dimensional data—geography, budgets, operating hours, and cultural context—the interface is engineered to abstract this complexity while ensuring the user remains the final decision-maker. The following subsections detail the high-fidelity wireframes that define the application’s interaction paradigms.

3.7.1 Visual Discovery and Personalized Retrieval

The application entry point (Figure 3.2, Left) is the Home Screen, which is divided into two distinct functional zones designed to balance exploration with personalization.

1. The Interactive Map Explorer (Top): Dominating the upper half of the screen is the Mapbox GL engine. Unlike standard navigation apps that center on the user’s physical GPS location, this view anchors to the *target destination* (e.g., Cairo). This "Exploration-First" design allows users to traverse the geospatial index intuitively, identifying clusters of activity and high-density areas without the cognitive load of text-based lists.

2. The Recommendation Feed (Bottom): Beneath the map lies the “Trending & Recommended” scroll view. This is not a static list; it is the visual output of the **User Profile Vector Search**. Upon loading, the backend queries the vector store for POIs that align with the user’s established interests (e.g., "Historical Sites" or "Budget Dining"). This section serves as the primary mechanism for serendipitous discovery, surfacing "hidden gems" that match the user’s persona but might not appear in a generic search.

3.7.2 Trust Calibration and The Ground Truth Interface

A central research objective of VOYO is solving the "Hallucination Problem" in generative AI. The **Info Card** (Figure 3.3) acts as the system’s "Ground Truth" interface—a trust calibration mechanism that separates verified facts from generative advice.

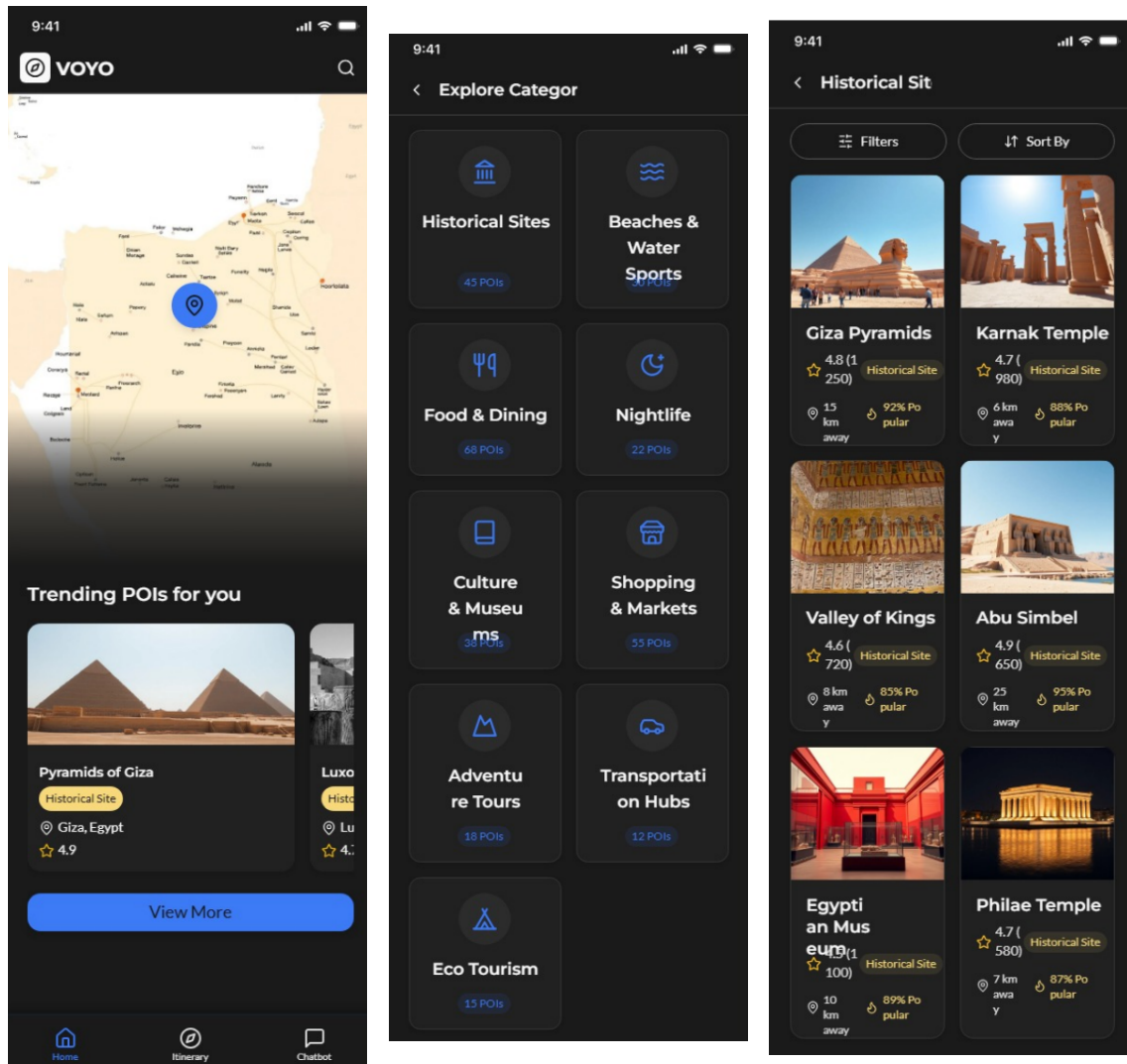
Immutable Data Presentation: While the descriptive text may be enriched by the LLM pipeline for readability, critical logistical data—specifically operating hours, ticket prices, and historical significance—is retrieved strictly from the relational database columns. The UI visually segregates these hard constraints in a rigid, standardized bottom sheet. This design signals to the user that this data is verified and immutable, distinct from the fluid conversation of the chatbot.

The Agentic Bridge: The Info Card also serves as the nexus between the static database and the dynamic agents.

- **Link to Planner:** The "Add to Itinerary" action triggers the Planner Agent (Agent B), which does not simply append the item but re-optimizes the entire schedule to accommodate the new stop’s time constraints.
- **Link to Cleo:** Users can initiate a context-aware chat directly from this card (e.g., "Ask Cleo about this"). This injects the specific POI’s metadata into Cleo’s context window, allowing for highly specific follow-up questions without re-prompting.

3.7.3 Dual-Agent Interaction Paradigms

The system’s multi-agent architecture necessitates two distinct user interface paradigms, as shown in Figure 3.4.



Home: Map & Feed

Taxonomy Filter

POI Catalogue

Figure 3.2: The Discovery Workflow. The Home Screen (Left) combines geospatial exploration with personalized vector recommendations. The Taxonomy (Center) and Catalogue (Right) views allow for structured, filter-based retrieval.

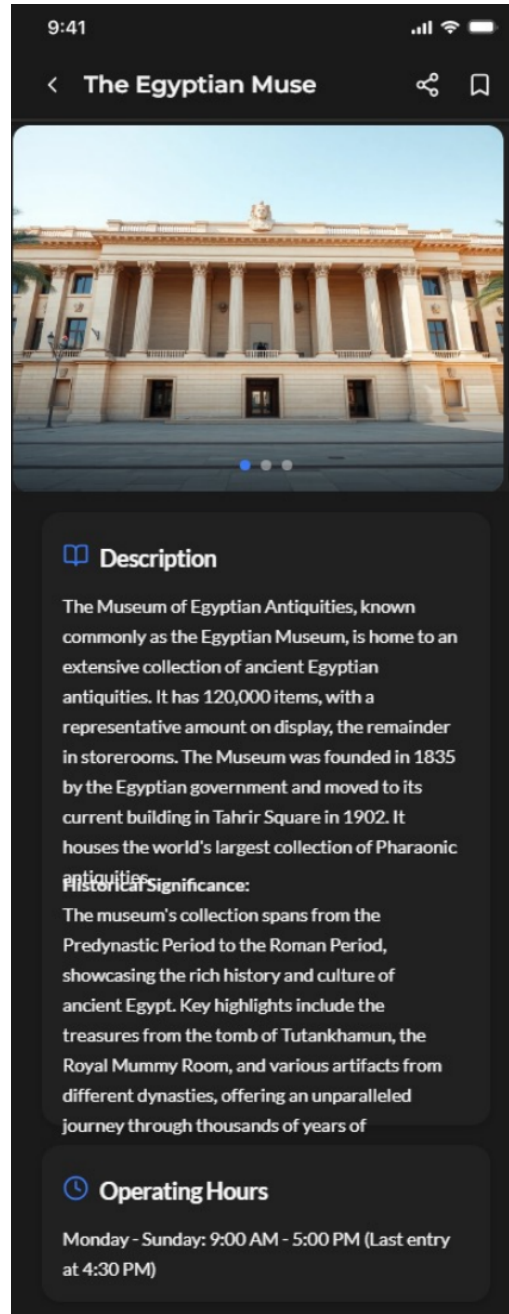


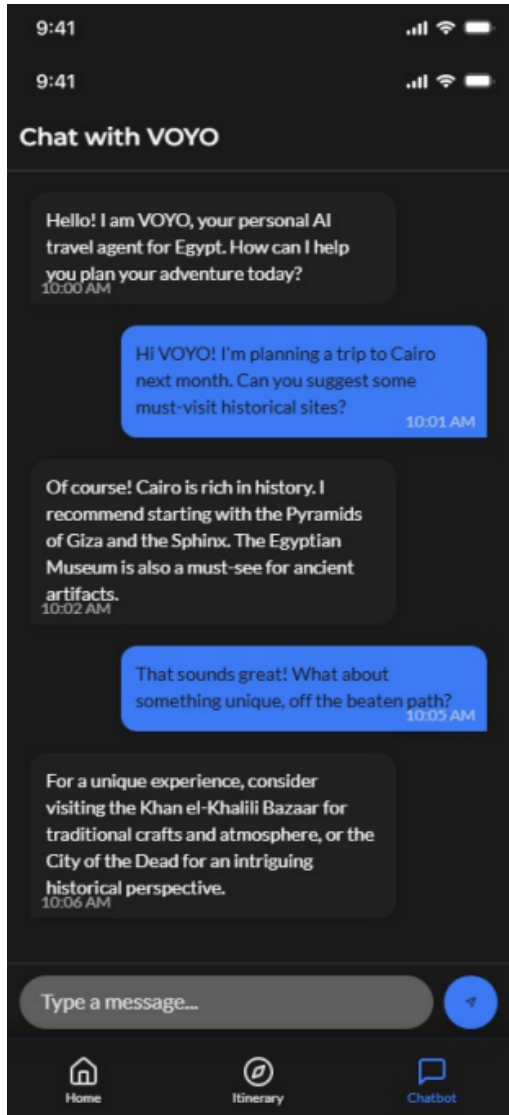
Figure 3.3: The Info Card (Ground Truth Interface). This view serves as a trusted data source, presenting verified hard constraints while providing direct context bridges to both the Planner Agent and the Cleo Chatbot.

3.7.3.1 The Conversational Interface (Cleo)

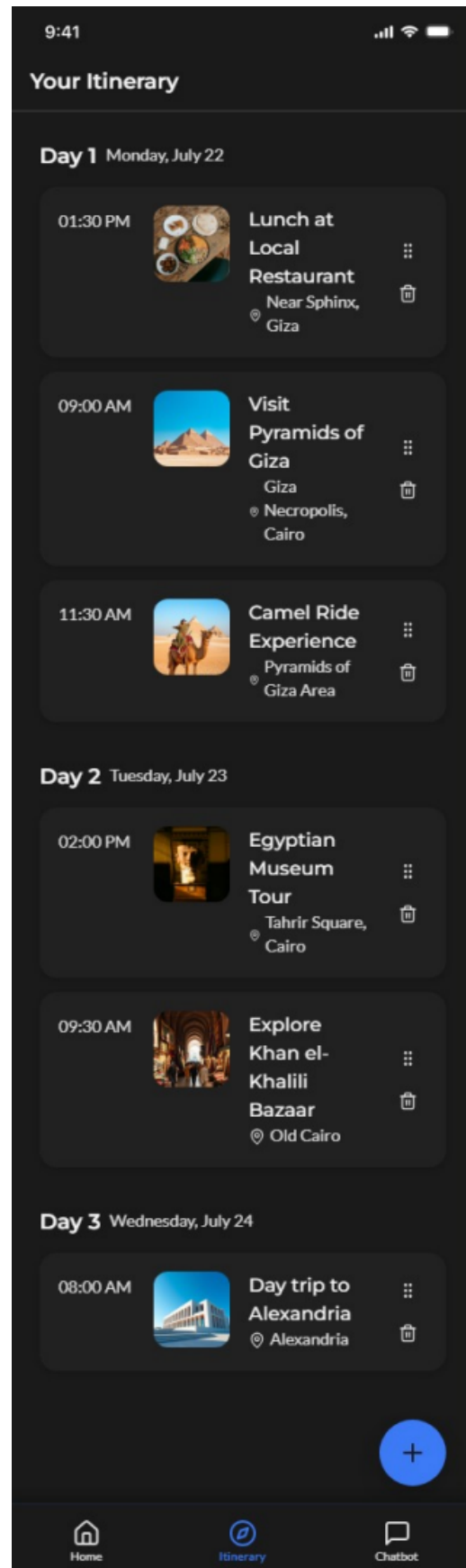
The **Cleo Interface** (Figure 3.4, Left) facilitates open-ended query resolution. It features streamed responses (Server-Sent Events) to minimize perceived latency. Crucially, this interface is "Context-Persistent"—it retains memory of the user's previous interactions across the map and itinerary screens, ensuring the persona remains consistent throughout the session.

3.7.3.2 The Structured Planning Interface (The Planner)

The **Itinerary View** (Figure 3.4, Right) is the visual manifestation of the Planner Agent's logic. Once the agent resolves the constraints via Chain-of-Thought reasoning, the output is parsed into a chronological timeline. This view supports "Human-in-the-Loop" editing: users can manually reorder or remove items, which sends a feedback signal to the backend to re-calculate travel times and valid opening hours dynamically.



Agent A: Cleo (Chat)



Agent B: Planner (Timeline)

Figure 3.4: Agentic Interfaces. The unstructured chat interface (Left) handles intent and queries, while the structured timeline (Right) visualizes the logical output of the Planner Agent.

Chapter 4

Results and Discussion

Chapter 5

Conclusion

References

- [1] R. Gupta et al. “The shift from models to compound ai systems.” Berkeley Artificial Intelligence Research Blog. [Online]. Available: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>
- [2] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024.
- [3] Q. Wu et al., “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [4] J. Xie et al., “Travelplanner: A benchmark for real-world planning with language agents,” *arXiv preprint arXiv:2402.01622*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01622>
- [5] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.
- [6] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” *arXiv preprint arXiv:2305.15334*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.15334>
- [7] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [8] C.-K. Pai, Y. Liu, S. Kang, and A. Dai, “The role of perceived smart tourism technology experience for tourist satisfaction, happiness and revisit intention,” *Sustainability*, vol. 12, p. 6592, 2020.
- [9] Y. Liu, H. Tan, G. Cao, and Y. Xu, “Enhancing user engagement through adaptive ui/ux design,” *Computer Science & IT Research Journal*, vol. 5, pp. 1942–1962, 2024.
- [10] K. D. Christina, S. S. Maesaroh, and M. D. A. Ridlo, “Ai-driven customer engagement in tokopedia: Satisfaction mediation on chatbot and recommendation system based on utaut,” *Journal of Business and Information Systems (e-ISSN: 2685-2543)*, vol. 7, no. 2, pp. 307–322, 2025.
- [11] H. Pang, Z. Hu, and L. Wang, “How perceived motivations influence user stickiness and sustainable engagement with ai-powered chatbots,” *Journal of Theoretical and Applied Electronic Commerce Research*, vol. 20, no. 3, p. 228, 2025.
- [12] E. E. Onuiri, H. C. Omoroje, C. G. Ntima, and A. A. Omotunde, “Intelligent tourism management system,” *American Scientific Research Journal for Engineering, Technology, and Sciences*, vol. 18, no. 1, pp. 304–315, 2016.

- [13] D. H. AlSaeed, “Locus: A mobile tourism application and recommender system for personalized places and activities,” *Informatica*, vol. 47, no. 2, 2023.
- [14] R.-H. Tsaih, H.-L. Chang, C.-C. Hsu, and D. C. Yen, “The ai tech-stack model,” *Communications of the ACM*, vol. 66, no. 3, pp. 69–77, 2023.
- [15] N. Swanepoel, “Development of an architecture and web-based demonstrator for tourist itinerary planning,” Ph.D. dissertation, Stellenbosch: Stellenbosch University, 2022.